Module 5

Advanced UVM Concepts

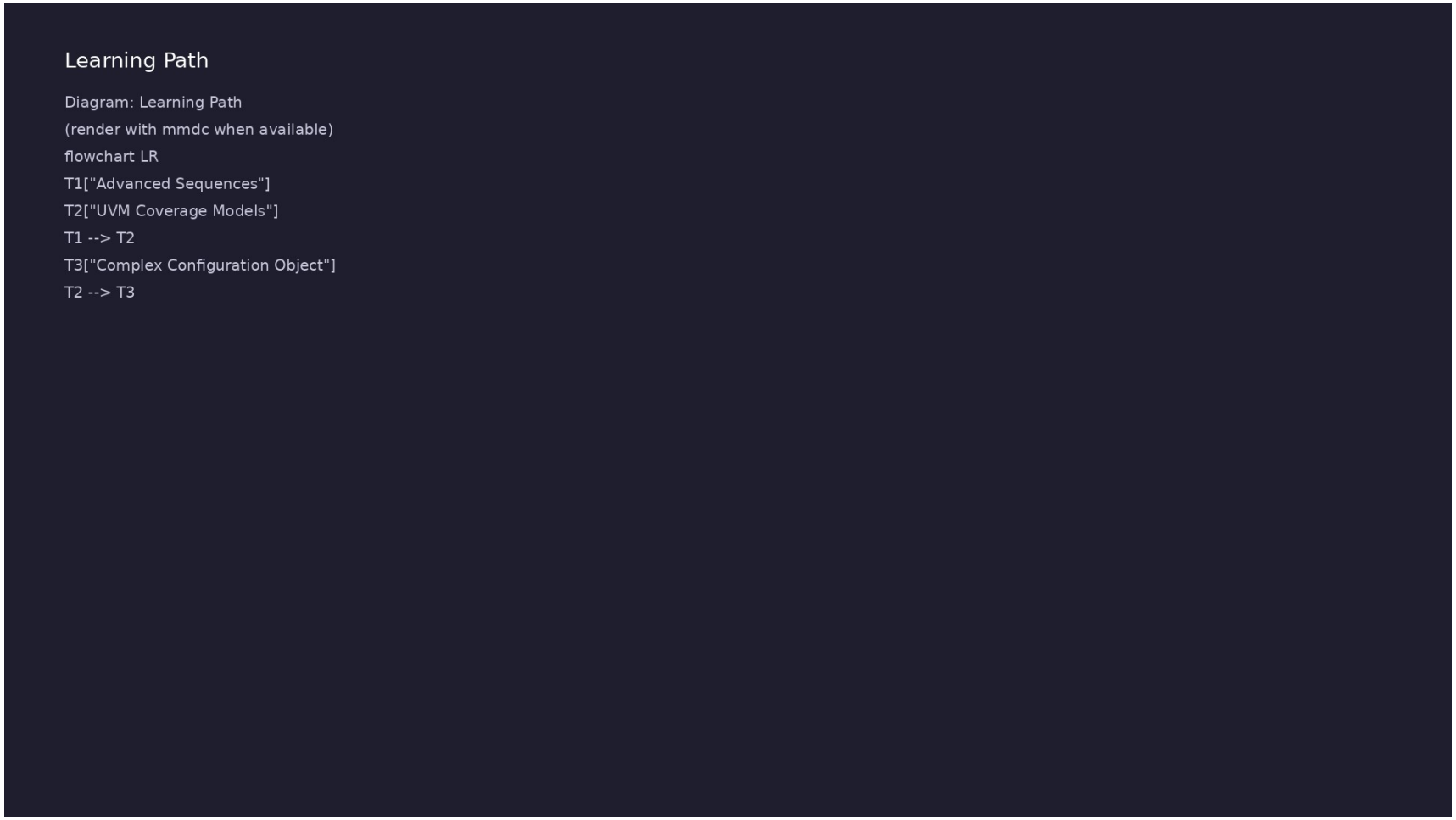
Learning objectives

- Master sequences, coverage, configuration, and virtual sequences

Prerequisites

- See module README

Learning path



Master sequences, coverage, configuration, and virtual sequences

Overview

- This module covers advanced UVM concepts including virtual sequences, coverage models, complex conf...

How to learn this module

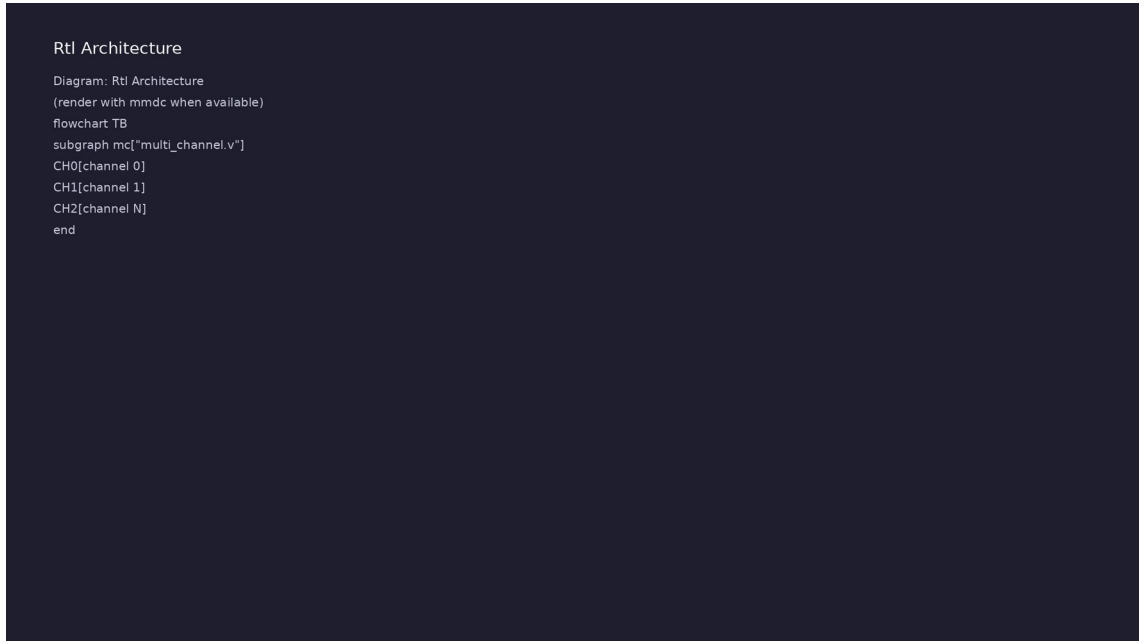
Suggested learning path

- Module 4 agent/scoreboard integration is prerequisite — multi-channel env extends the same TLM patt…
- Review `module5/dut/advanced/multi_channel.v` — multiple logical channels needing coordinated stimu…
- Study virtual sequencer concept before `virtual_sequences/` examples — one coordinator, many agents
- Skim RAL terminology: register map, fields, adapter, predictor — used in `register_model/`
- Coverage and callbacks modify observation and driver behavior without deep subclass trees

Design architecture

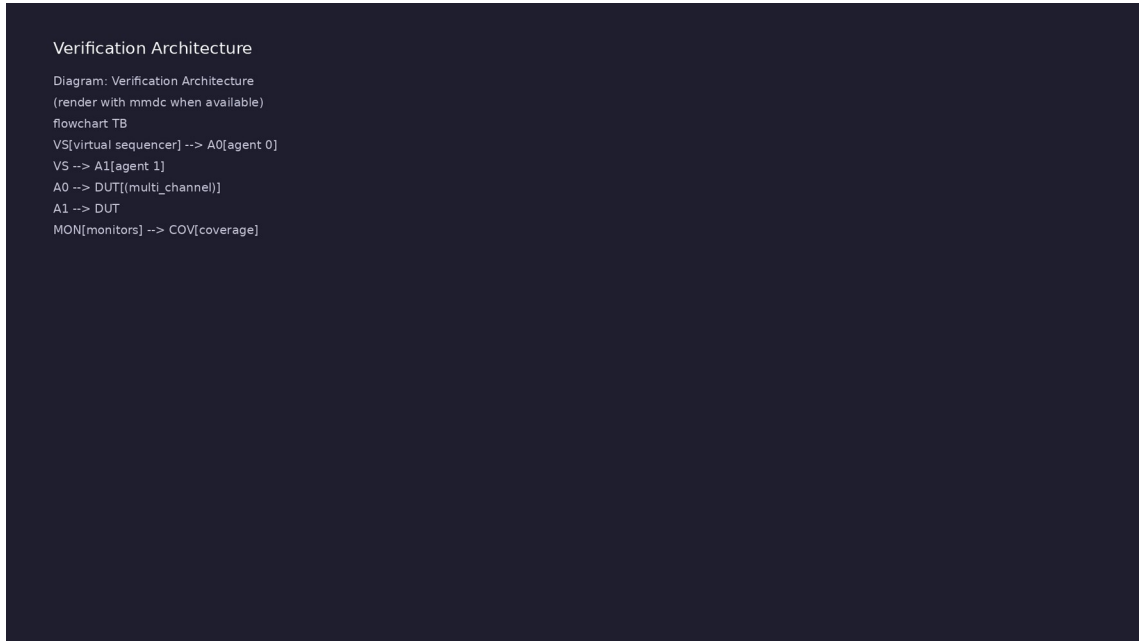
1. Advanced DUT architecture

- `module5/dut/advanced/multi_channel.v` — multiple logical channels for coordination labs
- Designed for virtual sequences and multi-stream scoreboarding
- Register abstraction layer examples align with memory-mapped control/status



2. Advanced UVM environment architecture

- Virtual sequencers coordinate multiple agents on different channels
- Coverage collectors sample transaction fields and cross coverage
- Callbacks extend driver/monitor without subclass explosion
- Configuration objects centralize knobs (active/passive, timeouts, enable flags)



3. Execution pipeline

- Build: multi-channel RTL plus env with multiple agents, virtual sequencer, coverage, register model
- Connect: per-channel monitor analysis → scoreboard/coverage; RAL adapter linked to bus sequencer
- Sim: virtual sequence launches child sequences in order; callbacks may alter driver/monitor behavio…
- Check: scoreboard per channel + coverage goals + register mirror consistency vs DUT state

4. Register model layer

- RAL-style maps: registers, fields, access policies (RW, RO, WO)
- Adapter links bus transactions to register reads/writes in tests
- Predictor/update paths keep mirror model consistent with DUT
- Register sequences replace raw bus wiggling for software-visible setup

DUT — multi-channel block

```
/**
 * Module 5: Multi-Channel Interface
 *
 * A multi-channel interface for advanced UVM testing.
 *
 * Ports:
 *   clk:         Clock signal
 *   rst_n:       Active-low reset
 *   master_valid: Master channel valid
 *   master_ready: Master channel ready
 *   master_data:  Master channel data
 *   slave_valid:  Slave channel valid
 *   slave_ready:  Slave channel ready
 *   slave_data:   Slave channel data
 */
```

```
# ...
```

Virtual sequencer pattern

```
"""
Module 5 Example 5.1: Virtual Sequences
Demonstrates virtual sequencer and virtual sequence coordination.
"""


from pyuvm import *
import cocotb
from cocotb.triggers import Timer

# Try to import uvm_seq_item_pull_port explicitly if not available from
pyuvm import *
try:
    # Check if already available
    _ = uvm_seq_item_pull_port
except NameError:
    # Try importing from TLM modules
    try:
```

Key files to study

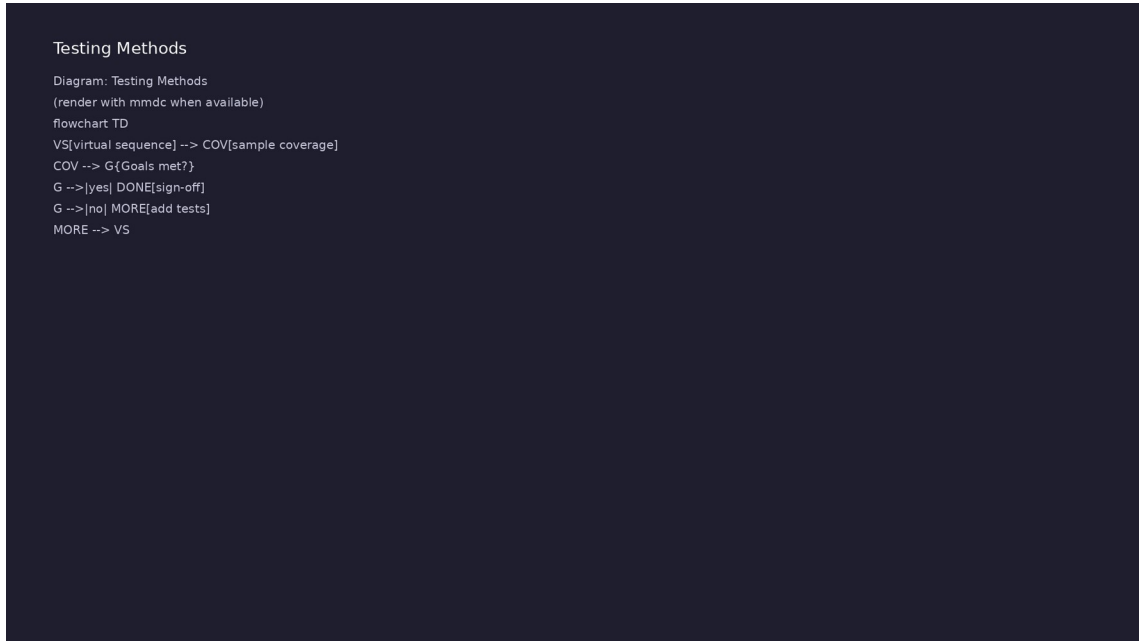
Open these in the repo

- `module5/dut/advanced/multi_channel.v` — multi-stream DUT for virtual sequence coordination
- `module5/examples/virtual_sequences/virtual_sequence_example.py` — child sequences on multiple sequ...
- `module5/examples/coverage/coverage_example.py` — functional coverage sampling hooks
- `module5/examples/register_model/register_model_example.py` — RAL map, adapter, mirror updates
- `module5/tests/pyuvm_tests/test_advanced_uvm.py` — integrated advanced env regression
- `scripts/module5.sh` — `--virtual-sequences`, `--coverage`, `--register-model`, ...

Verification & testing methods

1. Coverage-driven verification

- Functional coverage on
  transaction types, channel IDs,
  and corner field values
- Coverage goals gate regression
  sign-off in advanced flows
- Examples under
  `module5/examples/coverage/`
  show sampling hooks



2. Virtual sequences and configuration tests

- Virtual sequences start child sequences on multiple sequencers in order
- Config tests prove ConfigDB + config object overrides change behavior
- Callback tests inject errors or delays to validate robustness

3. Step-by-step lab execution

- 1. Virtual sequences: `./scripts/module5.sh --virtual-sequences`
- 2. Coverage/callbacks: `./scripts/module5.sh --coverage --callbacks`
- 3. Configuration: `./scripts/module5.sh --configuration`
- 4. Register model: `./scripts/module5.sh --register-model`
- 5. Integrated: `./scripts/module5.sh --pyuvm-tests` — all advanced mechanisms in one regression

4. Closure

- `./scripts/module5.sh --pyuvm-tests`; register model and callback demos in EXAMPLES.md
- Assessment: virtual sequences, coverage, callbacks, configuration, register model
- Demonstrate register read/write through RAL adapter rather than ad-hoc pin toggling

Syllabus topics

1. Advanced Sequences (1/4)

- Virtual Sequences
- What are virtual sequences?
- Virtual sequence purpose
- Multiple sequencer coordination
- Virtual sequence implementation
- Sequence Libraries

1. Advanced Sequences (2/4)

- Base sequence classes
- Derived sequences
- Sequence reuse patterns
- Sequence organization
- Sequence Arbitration
- Sequencer arbitration

1. Advanced Sequences (3/4)

- Priority mechanisms
- Lock and grab
- Sequence coordination
- Layered Sequences
- High-level sequences
- Low-level sequences

1. Advanced Sequences (4/4)

- Sequence composition
- Protocol layering

2. UVM Coverage Models (1/4)

- Coverage Overview
- What is coverage?
- Coverage types
- Coverage goals
- Coverage metrics
- Functional Coverage

2. UVM Coverage Models (2/4)

- Coverage models
- Coverage groups
- Coverpoints
- Coverage bins
- Coverage Implementation
- Coverage class structure

2. UVM Coverage Models (3/4)

- Coverage sampling
- Coverage analysis
- Coverage reporting
- Coverage Patterns
- Transaction coverage
- Protocol coverage

2. UVM Coverage Models (4/4)

- State coverage
- Cross coverage

3. Complex Configuration Objects (1/4)

- Configuration Objects
- Configuration class design
- Configuration fields
- Configuration methods
- Configuration validation
- Configuration Hierarchy

3. Complex Configuration Objects (2/4)

- Hierarchical configuration
- Configuration inheritance
- Configuration override
- Configuration patterns
- Resource Database
- Resource database usage

3. Complex Configuration Objects (3/4)

- Resource types
- Resource lookup
- Resource management
- Configuration Callbacks
- Configuration callbacks
- Pre/post callbacks

3. Complex Configuration Objects (4/4)

- Callback registration
- Callback execution

4. UVM Callbacks (1/4)

- Callback Overview
- What are callbacks?
- Callback purpose
- Callback types
- Callback benefits
- Callback Implementation

4. UVM Callbacks (2/4)

- Callback class definition
- Callback registration
- Callback execution
- Callback patterns
- Pre/Post Callbacks
- Pre-callbacks

4. UVM Callbacks (3/4)

- Post-callbacks
- Callback ordering
- Callback control
- Callback Use Cases
- Driver callbacks
- Monitor callbacks

4. UVM Callbacks (4/4)

- Scoreboard callbacks
- Test callbacks

5. UVM Register Model (Advanced) (1/5)

- Register Model Overview
- Register model purpose
- Register model structure
- Register model benefits
- Register model components
- Register Model Components

5. UVM Register Model (Advanced) (2/5)

- `uvm_reg_block` - Register blocks
- `uvm_reg` - Registers
- `uvm_reg_field` - Register fields
- `uvm_reg_map` - Address maps
- Register Operations
- Register read

5. UVM Register Model (Advanced) (3/5)

- Register write
- Register peek/poke
- Register update
- Register Sequences
- Register access sequences
- Register test sequences

5. UVM Register Model (Advanced) (4/5)

- Register model integration
- Register predictor
- Backdoor Access
- Backdoor read/write
- Backdoor vs frontdoor
- Backdoor use cases

5. UVM Register Model (Advanced) (5/5)

- Backdoor implementation

6. Virtual Sequences and Virtual Sequencers (1/3)

- Virtual Sequencer
- Virtual sequencer purpose
- Virtual sequencer structure
- Multiple sequencer references
- Virtual sequencer implementation
- Virtual Sequence Coordination

6. Virtual Sequences and Virtual Sequencers (2/3)

- Coordinating multiple sequencers
- Parallel sequence execution
- Sequence synchronization
- Sequence coordination patterns
- Virtual Sequence Patterns
- Master-slave coordination

6. Virtual Sequences and Virtual Sequencers (3/3)

- Multi-channel coordination
- Protocol coordination
- Test coordination

7. Coverage Analysis and Closure (1/3)

- Coverage Analysis
- Coverage collection
- Coverage reporting
- Coverage gaps
- Coverage analysis tools
- Coverage Closure

7. Coverage Analysis and Closure (2/3)

- Coverage goals
- Coverage strategies
- Coverage improvement
- Coverage metrics
- Coverage Patterns
- Functional coverage

7. Coverage Analysis and Closure (3/3)

- Code coverage
- Assertion coverage
- Coverage correlation

8. Advanced Configuration Patterns (1/2)

- Configuration Strategies
- Top-down configuration
- Bottom-up configuration
- Mixed configuration
- Configuration best practices
- Dynamic Configuration

8. Advanced Configuration Patterns (2/2)

- Runtime configuration
- Configuration updates
- Configuration validation
- Configuration debugging

9. Performance Optimization (1/2)

- Testbench Performance
- Performance bottlenecks
- Optimization strategies
- Memory optimization
- Simulation speed
- Sequence Optimization

9. Performance Optimization (2/2)

- Efficient sequence design
- Sequence reuse
- Sequence caching
- Sequence performance

10. Advanced Debugging Techniques (1/2)

- UVM Debugging
- UVM debugging tools
- Phase debugging
- Component debugging
- Transaction debugging
- Coverage Debugging

10. Advanced Debugging Techniques (2/2)

- Coverage gaps
- Coverage analysis
- Coverage improvement
- Coverage tools

Command reference highlights

Advanced UVM topics

- `./scripts/module5.sh --virtual-sequences --configuration` — coordination and config objects
- `./scripts/module5.sh --coverage --callbacks` — coverage sampling and callback injection
- `./scripts/module5.sh --register-model` — RAL map and bus adapter drill

Integrated regression

- `./scripts/module5.sh --pyuvm-tests` — `test_advanced_uvm` against multi-channel DUT
- `cd module5/tests/pyuvm_tests && make SIM=verilator TEST=test_advanced_uvm`
- `./scripts/module5.sh` — all example tracks then tests when doing full module sweep

Coverage and config debug

- Inspect coverage reports/sample counts from example logs when goals are not met
- Config object + ConfigDB overrides in `--configuration` — re-run to verify behavior change
- Callback tests inject delays/errors — use verbosity to see callback invocation order

Hands-on examples

Module 5 orchestrator

```
$ ./scripts/module5.sh --help
```

Terminal: module5_check

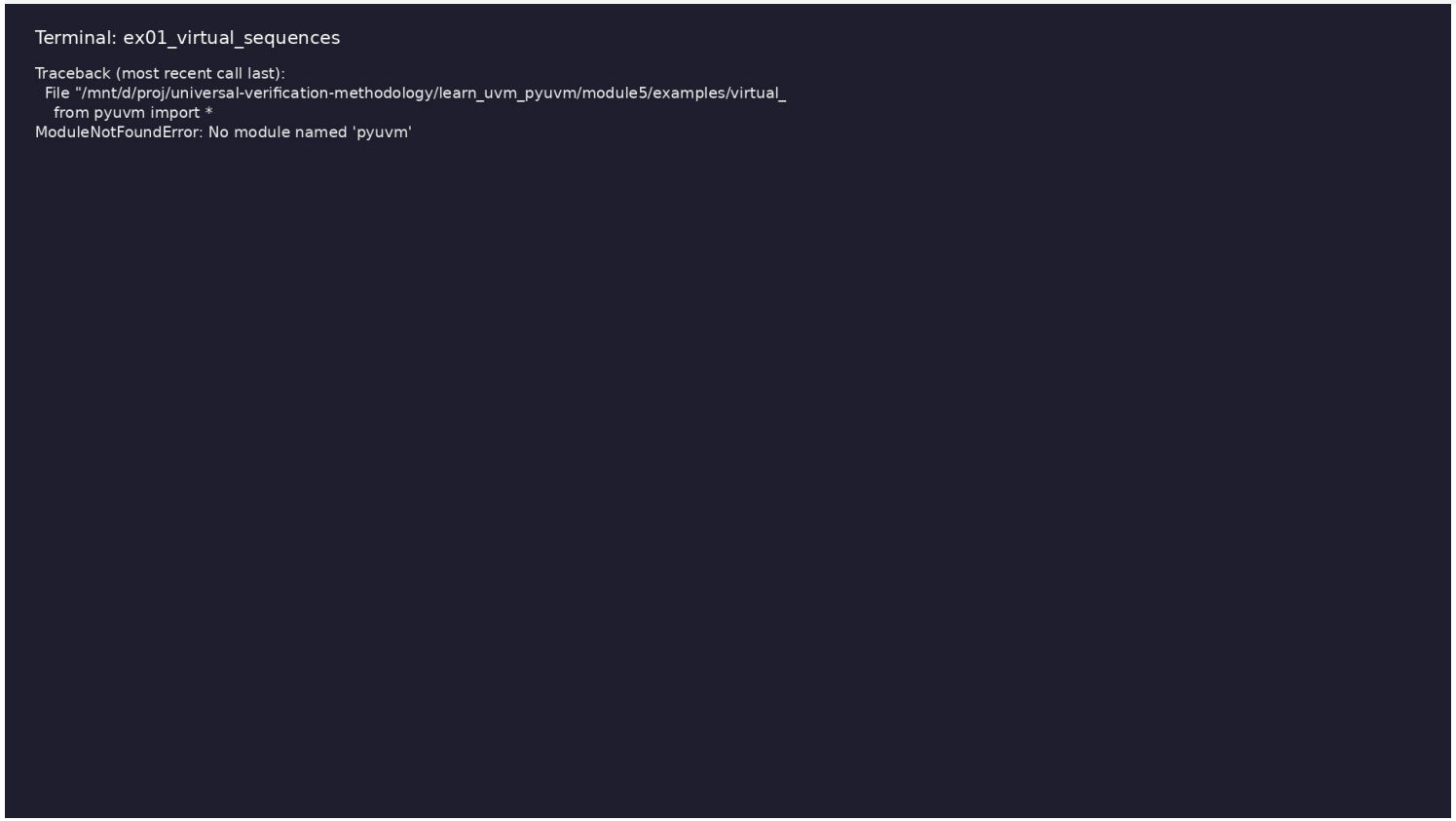
```
[0;36m==========================================[0m
[0;36mModule 5: Advanced UVM Concepts[0m
[0;36m==========================================[0m

Usage: ./scripts/module5.sh [OPTIONS]

Module 5: Advanced UVM Concepts
This script runs examples and tests for Module 5.

OPTIONS:
    Examples:
        --virtual-sequences  Run virtual sequence examples
        --coverage           Run coverage model examples
        --configuration      Run configuration object examples
        --callbacks          Run callback examples
        --register-model     Run register model examples
        --all-examples       Run all examples (default)
        --skip-examples      Skip all examples

    Tests:
        --pyuvm-tests        Run pyuvm tests

    Environment:
        --venv DIR           Virtual environment directory (default: .venv)
        --no-venv            Don't use virtual environment
        --sim SIMULATOR      Simulator to use (default: verilator)

    Other:
        --help, -h           Show this help message

EXAMPLES:
    # Run all examples
    ./scripts/module5.sh

    # Run specific examples
```

Exercise scaffold

```
./scripts/module5.sh --scaffold
```

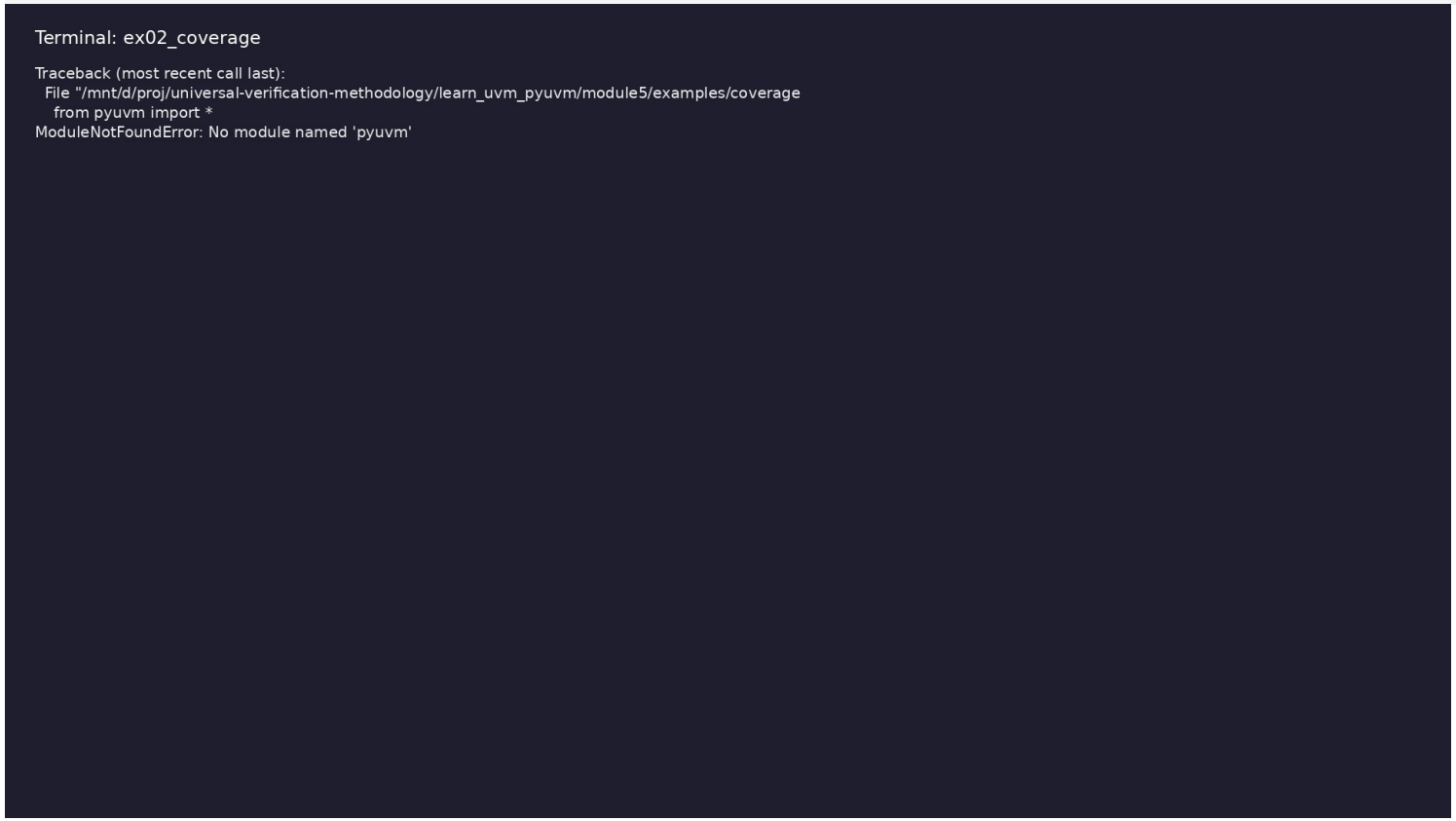
Demo: Virtual Sequences

```
$ .venv/bin/python
      module5/examples/virtual_sequences/virtual_sequence_example.py
```



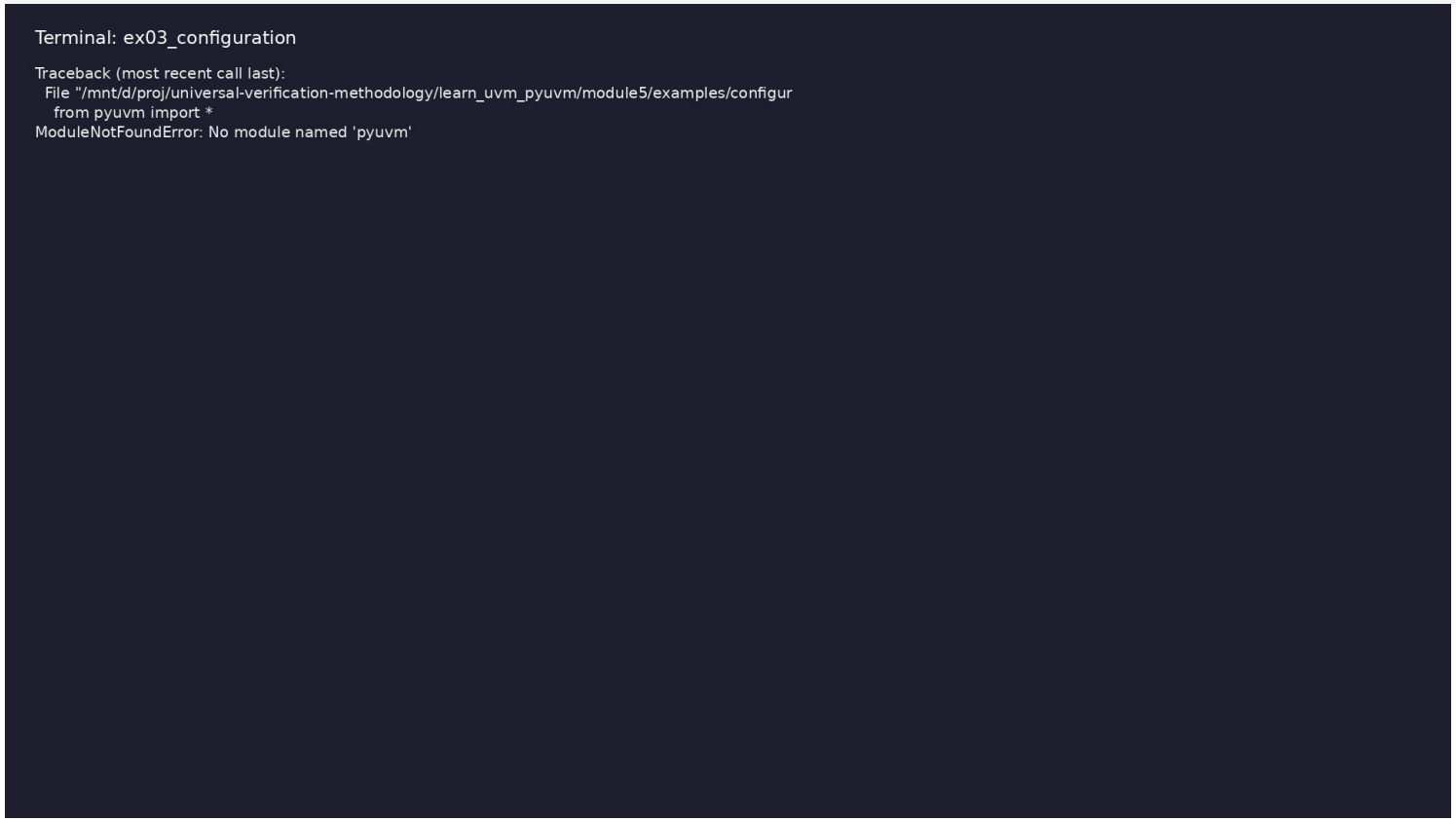
Demo: Coverage Models

```
$ .venv/bin/python module5/examples/coverage/coverage_example.py
```



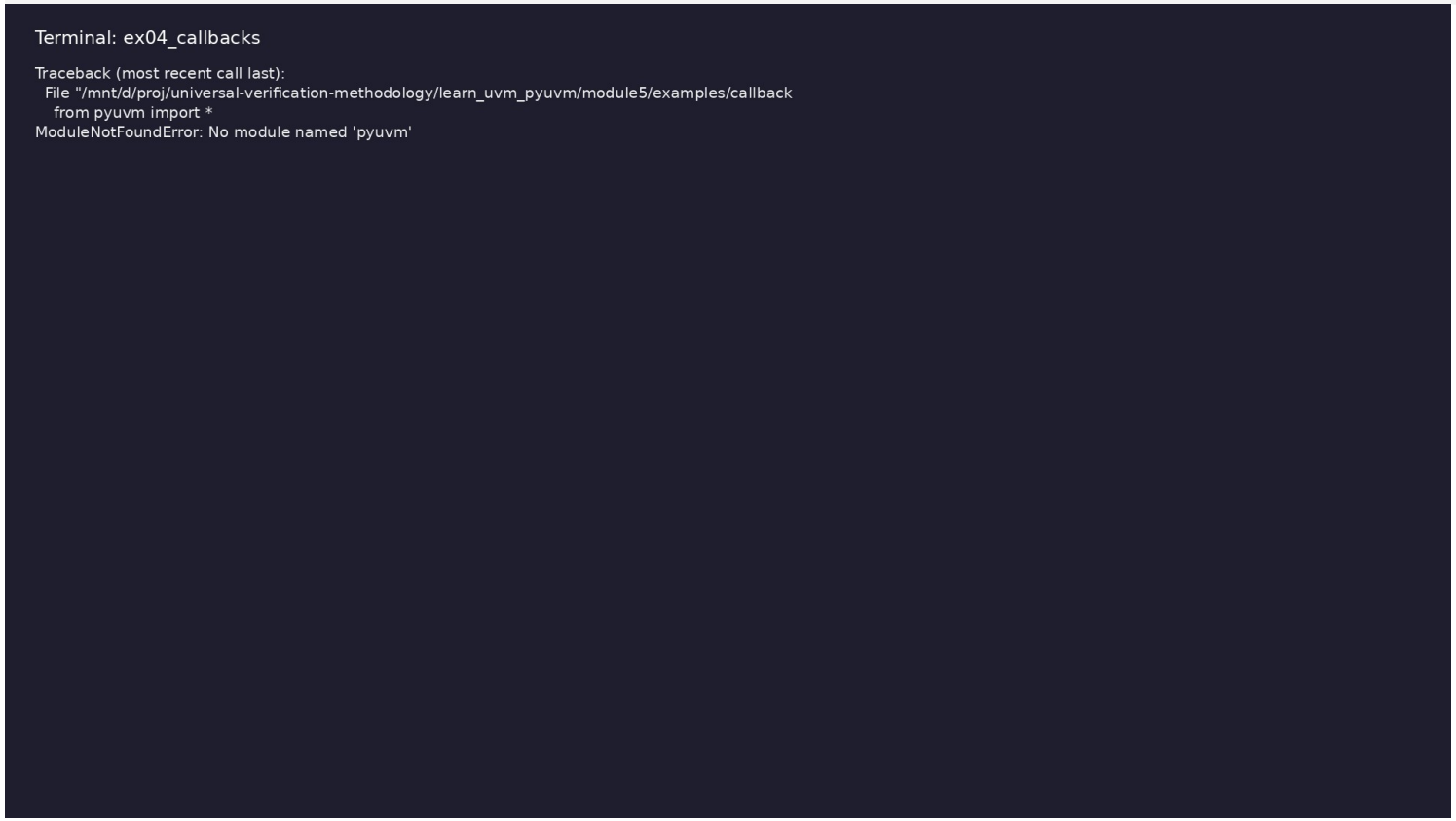
Demo: Configuration Objects

```
$ .venv/bin/python
      module5/examples/configuration/configuration_example.py
```



Demo: UVM Callbacks

```
$ .venv/bin/python module5/examples/callbacks/callback_example.py
```



Demo: Register Model

```
$ .venv/bin/python
      module5/examples/register_model/register_model_example.py
```

Practice & assessment

What you should know (1/3)

- Create and use virtual sequences
- Implement coverage models
- Design complex configuration objects
- Use UVM callbacks effectively
- Use advanced register model features
- Coordinate multiple sequencers

What you should know (2/3)

- Analyze and close coverage
- Optimize testbench performance
- Debug advanced testbenches
- Apply advanced patterns
- Virtual sequencer
- Virtual sequence

What you should know (3/3)

- Multiple sequencer coordination

Exercises

- Virtual Sequences
- Coverage Implementation
- Configuration Design
- Callback Implementation
- Register Model

Assessment checklist

- Can create virtual sequences
- Can implement coverage models
- Can design configuration objects
- Can use callbacks effectively
- Can use advanced register model
- Can coordinate multiple sequencers

Summary & next steps

- Master sequences, coverage, configuration, and virtual sequences
- Complete module5/CHECKLIST.md
- Review module5/EXAMPLES.md and run each lab
- Next: Next module in course